An

Industrial Training Report

on

Full Stack Web Development

Submitted in partial fulfillment for the award of degree of

Bachelor of Technology

In

Computer Science and Engineering

Submitted to:

Dr. Pradeep Jha

HOD-CSE/IT/AI&DS/IOT/CYBER SECURITY Dept.

Submitted By:

Milan Soni

22EGJCS141

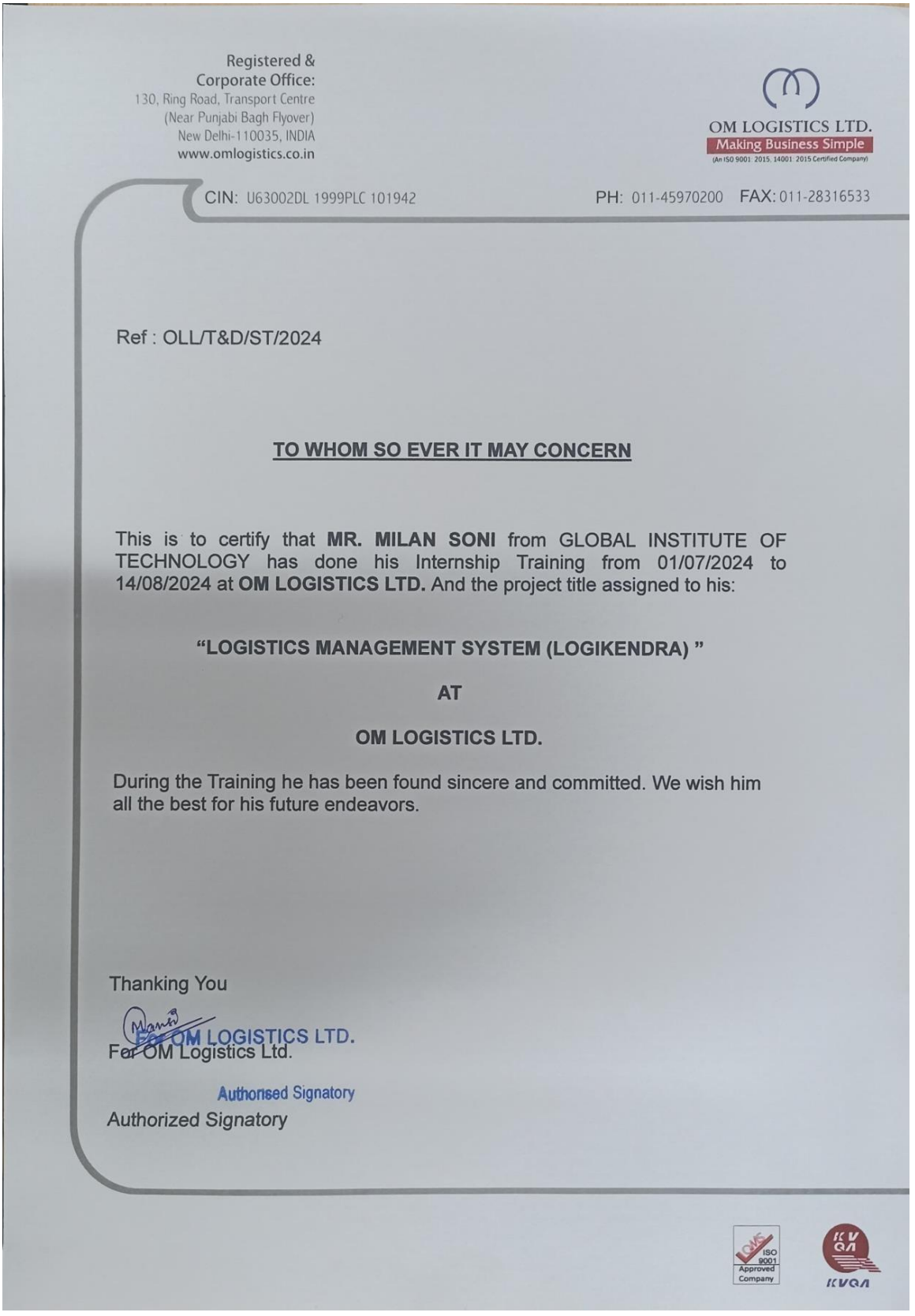
Department of Computer Science & Engineering

Global Institute of Technology

Jaipur (Rajasthan)-302022

Session: 2024-25

Certificate



Registered &
Corporate Office:
130, Ring Road, Transport Centre
(Near Punjabi Bagh Flyover)
New Delhi-110035, INDIA
www.omlogistics.co.in

OM LOGISTICS LTD.
Making Business Simple
(An ISO 9001: 2015, 14001: 2015 Certified Company)

CIN: U63002DL 1999PLC 101942 PH: 011-45970200 FAX: 011-28316533

Ref : OLL/T&D/ST/2024

TO WHOM SO EVER IT MAY CONCERN

This is to certify that **MR. MILAN SONI** from GLOBAL INSTITUTE OF TECHNOLOGY has done his Internship Training from 01/07/2024 to 14/08/2024 at **OM LOGISTICS LTD.** And the project title assigned to his:

"LOGISTICS MANAGEMENT SYSTEM (LOGIKENDRA) "

AT

OM LOGISTICS LTD.

During the Training he has been found sincere and committed. We wish him all the best for his future endeavors.

Thanking You

For OM LOGISTICS LTD.

For OM Logistics Ltd.

Authorised Signatory

Authorized Signatory

ISO 9001 Approved Company

IRQA

Acknowledgement

I take this opportunity to express my deep sense of gratitude to my ITS coordinator **Mr. Hemant Mittal** Assistant Professor and **Mrs. Kangana Saini** Assistant Professor Department of Computer Science and Engineering, Global Institute of Technology, Jaipur, for his valuable guidance and cooperation throughout the Practical Training work. She provided constant encouragement and unceasing enthusiasm at every stage of the PracticalTraining work. I am grateful to our respected **Dr. I. C. Sharma**, Principal GIT for guidingme during Industrial Training period I express my indebtedness to **Dr. Pradeep Jha**, Headof Department of Computer Science and Engineering, Global Institute of Technology, Jaipur forproviding me ample support during my Industrial Training period.

Without their support and timely guidance, the completion of our Industrial Training wouldhave seemed a farfetched dream. In this respect we find ourselves lucky to have mentors of such a great potential.

Place: GIT, Jaipur

Milan Soni

22EGJCS141

B.Tech. V Semester, III Year, CSE

Abstract

This report delves into the dynamic and ever-evolving landscape of web development, exploring key trends, technologies, and best practices that have emerged to shape the waywebsites and web applications are designed and built. With the digital world experiencing unprecedented growth and transformation, web development has become a critical aspect of modern business and user engagement strategies. The report begins by highlighting theimportance of responsive web design and the impact of mobile-first approaches in ensuringa seamless user experience across various devices. It discusses the significance of user- centered design principles and the use of intuitive navigation and accessible web content toenhance user satisfaction and accessibility compliance.

In addition, the report delves into the role of modern web development frameworks and libraries, such as React, Angular, and Vue.js, in simplifying the development process and enhancing the performance of web applications. Furthermore, it explores the use ofRESTful and GraphQL APIs to facilitate data exchange between the client and server, enabling real-time communication and efficient data retrieval. Security is a paramount concern in web development, and this report examines the importance of implementing robust security measures, including SSL certificates, authentication protocols, and regular vulnerability assessments, to protect websites and web applications from cyber threats.

TABLE OF CONTENT

LIST OF FIGURES

LIST OF TABLES

CHAPTER 1

Introduction to Full Stack development

1.1 Full Stack Development

Web development is the work involved in developing a website for the Internet (World Wide Web) or an intranet (a private network).[1] Web development can range from developing a simple single static page of plain text to complex web applications, electronic businesses, and social network services. A more comprehensive list of tasks to which Web development commonly refers, may include Web engineering, Web design, Web content development, client liaison, client-side/server-side scripting, Web server and network security configuration, and e-commerce development.

1.2 Web Developer

Web development may be a collaborative effort between departments rather than the domain of a designated department. There are three kinds of Web developer specialization: front-end developer, back-end developer, and full-stack developer.

1.2.1 Front-end development

Front-end web development is the development of the graphical user interface of a website, through the use of HTML, CSS, and JavaScript, so that users can view and interact with that website.

1.2.2 Back-end development

Back-end development means working on server-side software, which focuses on everything you can't see on a website. Back-end developers ensure the website performs correctly, focusing on databases, back-end logic, application programming interface (APIs), architecture, and servers. On the other hand, back-end development deals with the server-side aspects that power websites and web applications. This includes database management

1.2.3 Full-stack development

A full stack web development means to develop both client and server software.

1.3 Website

A website (also written as a web site) is a collection of web pages and related content that is identified by a common domain name and published on at least one web server. Examples of notable websites are Google, Facebook, Amazon, and Wikipedia. All publicly accessible websites collectively constitute the World Wide Web. There are also private websites that can only be accessed on a private network, such as a company's internal website for its employees. Websites are typically dedicated to a particular topic or purpose, such as news, education, commerce, entertainment, or social networking. Hyperlinking between web pages guides the navigation of the site, which often starts with a home page.Users can access websites on a range of devices, including desktops, laptops, tablets, and smartphones. The app used on these devices is called a web browser.

1.4 Layers of website

When you're creating a web page, its structure should be relegated to your HTML, visual styles to the CSS, and behaviors to scripts.

1.4.1 HTML: The Structure Layer

The structure or content layer of a web page is the underlying HTML code of that page. Just as a house's frame creates a strong foundation upon which the rest of the house is built, a solid foundation of HTML creates a platform upon which a website can be created.

The structure layer is where you store all the content that your customers want to read or look at. HTML structure can consist of text and images, and it includes the hyperlinks that visitors will use to navigate around the website. This information is coded in standards-compliant HTML5 and can include text, images, and multimedia (video, audio, etc.).Every aspect of a site's content should be represented in the structure layer. This separation allows customers who have JavaScript turned off or who can't view CSS access to the entire website, if not all of its functionality.

1.4.2 CSS: The Styles Layer

This layer dictates how a structured HTML document will look to a site's visitors and is

defined by CSS (Cascading Style Sheets). These files contain stylistic instructions for how the document should be displayed in a web browser. The style layer usually includes media queries that change a site's display based on screen size and device.

All visual styles for a website should reside in an external stylesheet. You can use multiple stylesheets, but every CSS file requires an HTTP request to fetch it, affecting site performance.

1.4.3 JavaScript: The Behavior Layer

The behavior layer makes a website interactive, allowing the page to respond to user actions or to change based on a set of conditions. JavaScript is the most commonly used language for the behavior layer, but CGI and PHP are very frequently used, too.

When developers refer to the behavior layer, most of them mean the layer that is activated directly in the web browser. Use this layer to interact directly with the Document Object Model. Writing valid HTML in the content layer is important for DOM interactions in the behavior layer. When you build in the behavior layer, you should use external script files, just as with CSS, to optimize speed and performance.

Fig1 : Dynamic website

CHAPTER 2

HTML-OVERVIEW

HTML stands for Hyper Text Markup Language, which is the most widely used language on the Web to develop web pages. HTML was created by Berners-Lee in late 1991 but "HTML 2.0" was the first standard HTML specification which was published in 1995. HTML 4.01 was a major version of HTML and it was published in late 1999. Though the HTML 4.01 version is widely used, currently we are having the HTML-5 version which is an extension to HTML 4.01, and this version was published in 2012.

2.1 Why to Learn HTML?

Originally, HTML was developed with the intent of defining the structure of documents like headings, paragraphs, lists, and so forth to facilitate the sharing of scientific information between researchers. Now, HTML is being widely used to format web pages with the help of different tags available in HTML language.

HTML is a MUST for students and working professionals to become a great Software Engineer specially when they are working in Web Development Domain. I will list down some of the key advantages of learning HTML:

- Create Web site - You can create a website or customize an existing web template if you know HTML well.
- Become a web designer - If you want to start a career as a professional web designer, HTML and CSS designing is a must skill.
- Understand web - If you want to optimize your website, to boost its speed and performance, it is good to know HTML to yield best results.
- Learn other languages - Once you understand the basics of HTML then other related technologies like javascript, php, or angular become easier to understand.

2.2 Hello World using HTML

Just to give you a little excitement about HTML, I'm going to give you a small conventional HTML Hello World program.

```
<!DOCTYPE html>
```

```
<html>
<head>
<title>This is document title</title>
</head>
<body>
<h1>This is a heading</h1>
<p>Hello World!</p>
</body>
</html>
```

2.3 Applications of HTML

As mentioned before, HTML is one of the most widely used languages over the web. I'm going to list few of them here:

- Web pages development - HTML is used to create pages which are rendered over the web. Almost every page of the web has html tags in it to render its details in the browser.
- Internet Navigation - HTML provides tags which are used to navigate from one page to another and is heavily used in internet navigation.
- Responsive UI - HTML pages now-a-days works well on all platforms, mobile, tabs, desktop or laptops owing to responsive design strategy.
- Offline support HTML pages once loaded can be made available offline on the machine without any need of the internet.
- Game development- HTML5 has native support for rich experience and is now useful in the gaming development arena as well.

2.4 Prerequisites

Before proceeding with this tutorial you should have a basic working knowledge with Windows or Linux operating system, additionally you must be familiar with -

- Experience with any text editor like notepad, notepad++, or Edit plus etc.
- How to create directories and files on your computer.

- How to navigate through different directories.
- How to type content in a file and save them on a computer.
- Understanding about images in different formats like JPEG, PNG format

HTML- Basic tags

2.5 Heading Tag

Any document starts with a heading. You can use different sizes for your headings. HTML also has six levels of headings, which use the elements **<h1>, <h2>, <h3>, <h4>, <h5>,** and **<h6>**. While displaying any heading, the browser adds one line before and one line after that heading.

Example

```
<!DOCTYPE html>
<html>

<head>
<title>Heading Example</title>
</head>

<body>
<h1>This is heading 1</h1>
<h2>This is heading 2</h2>
<h3>This is heading 3</h3>
<h4>This is heading 4</h4>
<h5>This is heading 5</h5>
<h6>This is heading 6</h6>
</body>
</html>
```

2.6 Paragraph Tag

The ** ** tag offers a way to structure your text into different paragraphs. Each paragraph of text should go in between an opening and a closing tag as shown

below in the example −

Example

```
<!DOCTYPE html>
<html>
<head>
<title>Paragraph Example</title>
</head>

<body>
<p>Here is the first paragraph of text.</p>
<p>Here is a second paragraph of text.</p>
<p>Here is a third paragraph of text.</p>
</body>
</html>
```

2.7 Line Break Tag

Whenever you use the **
** element, anything following it starts from the next line. This tag is an example of an **empty** element, where you do not need opening and closing tags, as there is nothing to go in between them.

The
 tag has a space between the characters **br** and the forward slash. If you omit this space, older browsers will have trouble rendering the line break, while if you miss the forward slash character and just use
 it is not valid in XHTML.

Example

```
<!DOCTYPE html>
<html>
<head>
<title>Line Break Example</title>
</head>

<body>
```

```
<p>Hello<br />
You delivered your assignment on time.<br /> Thanks<br />
Mahnaz</p>
</body>
</html>
```

2.8 Centering Content

You can use ** ** tag to put any content in the center of the page or any table cell.

Example

```
<!DOCTYPE html>
<html>
<head>
<title>Centring Content Example</title>
</head>
<body>
<p>This text is not in the center.</p>
<center>
<p>This text is in the center.</p>
</center>
</body>
</html>
```

2.9 Horizontal Lines

Horizontal lines are used to visually break-up sections of a document. The **<hr>** tag creates a line from the current position in the document to the right margin and breaks the line accordingly.

For example, you may want to give a line between two paragraphs as in the given example below −

Example

```
<!DOCTYPE html>
<html>
<head>
<title>Horizontal Line Example</title>
</head>
<body>
<p>This is paragraph one and should be on top</p>
<hr />
<p>This is paragraph two and should be at bottom</p>
</body>
</html>
```

Again **<hr />** tag is an example of the **empty** element, where you do not need opening and closing tags, as there is nothing to go in between them.

The **<hr />** element has a space between the characters **hr** and the forward slash. If you omit this space, older browsers will have trouble rendering the horizontal line, while if you miss the forward slash character and just use **<hr>** it is not valid in XHTML.

2.10 Non breaking Spaces

Suppose you want to use the phrase "12 Angry Men." Here, you would not want a browser to split the "12, Angry" and "Men" across two lines −
An example of this technique appears in the movie "12 Angry Men."

In cases where you do not want the client browser to break text, you should use a non-breaking space entity ** ** instead of a normal space. For example, when coding the "12 Angry Men" in a paragraph, you should use something similar to the following code −

Example

```
<!DOCTYPE html>
<html>
<head>
<title>Non breaking Spaces Example</title>
```

```
</head>

<body>
<p>An example of this technique appears in the movie "12  Angry
 Men."</p>
</body>
</html>
```

HTML-ELEMENTS AND ATTRIBUTES

2.11 HTML Element

An **HTML element** is defined by a starting tag. If the element contains other content, it ends with a closing tag, where the element name is preceded by a forward slash as shown below with few tags −

Start Tag	Content	End Tag
	This is paragraph content.	
<h1>	This is heading content.	</h1>
	This is division content.	
 	This is breaking content.	</br>

Table 1: HTML element

So here ** .... ** is an HTML element, **<h1>...</h1>** is another HTML element. There are some HTML elements which don't need to be closed, such as **<img.../>**, **<hr />** and **
** elements. These are known as **void elements**.

HTML documents consist of a tree of these elements and they specify how HTML documents should be built, and what kind of content should be placed in what part of an HTML document.

2.12 HTML Tag vs. Element

An HTML element is defined by a *starting tag*. If the element contains other content, it ends with a closing tag.

For example, ** ** is the starting tag of a paragraph and ** ** is the closing tag of the same paragraph but ** This is paragraph ** is a paragraph element.

2.13 Nested HTML Elements

It is very much allowed to keep one HTML element inside another HTML element −

Example

```
<!DOCTYPE html>
<html>
<head>
<title>Nested Elements Example</title>
</head>
<body>
<h1>This is <i>italic</i> heading</h1>
<p>This is <u>underlined</u> paragraph</p>
</body>
</html>
```

2.14 Core Attributes

The four core attributes that can be used on the majority of HTML elements (although not all) are −

- Id
- Title
- Class
- Style

❖ The Id Attribute

The **id** attribute of an HTML tag can be used to uniquely identify any element within an HTML page. There are two primary reasons that you might want to use an id attribute on

an element −

- If an element carries an id attribute as a unique identifier, it is possible to identify just that element and its content.
- If you have two elements of the same name within a Web page (or style sheet), you can use the id attribute to distinguish between elements that have the same name.
- We will discuss the style sheet in a separate tutorial. For now, let's use the id attribute to distinguish between two paragraph elements as shown below.

Example

```
<p id = "html">This para explains what is HTML</p>

<p id = "css">This para explains what is Cascading Style Sheet</p>
```

❖ The title Attribute

The **title** attribute gives a suggested title for the element. They syntax for the **title** attribute is similar as explained for **id** attribute −

The behavior of this attribute will depend upon the element that carries it, although it is often displayed as a tooltip when the cursor comes over the element or while the element is loading.

Example

```
<!DOCTYPE html>

<html>

<head>

<title>The title Attribute Example</title>

</head>

<body>

<h3 title = "Hello HTML!">Titled Heading Tag Example</h3>

</body>

</html>
```

Now try to bring your cursor over "Titled Heading Tag Example" and you will see that whatever title you used in your code is coming out as a tooltip of the cursor.

❖ The class Attribute

The **class** attribute is used to associate an element with a style sheet, and specifies the class of element. You will learn more about the use of the class attribute when you will learn Cascading Style Sheets (CSS). So for now you can avoid it.
The value of the attribute may also be a space-separated list of class names. For example − class = "className1 className2 className3"

❖ The style Attribute

The style attribute allows you to specify Cascading Style Sheet (CSS) rules within the element.

```
<!DOCTYPE html>
<html>
<head>
<title>The style Attribute</title>
</head>
<body>
<p style = "font-family:arial; color:#FF0000;">Some text...</p>
</body>
</html>
```

At this point of time, we are not learning CSS, so just let's proceed without bothering much about CSS. Here, you need to understand what are HTML attributes and how they can be used while formatting content.

2.15 Internationalization Attributes

There are three internationalization attributes, which are available for most (although not all) XHTML elements.

- dir
- lang
- xml:lang

❖ The dir Attribute

The **dir** attribute allows you to indicate to the browser about the direction in which the text should flow. The dir attribute can take one of two values, as you can see in the table that follows −

Value	Meaning
ltr	Left to right (the default value)
rtl	Right to left (for languages such as Hebrew or Arabic that are read right to left)

Example

```
<!DOCTYPE html>
<html dir = "rtl">
<head>
<title>Display Directions</title>
</head>
<body>
This is how IE 5 renders right-to-left directed text.
</body>
</html>
```

When *dir* attribute is used within the <html> tag, it determines how text will be presented within the entire document. When used within another tag, it controls the text's direction for just the content of that tag.

❖ The lang Attribute

The **lang** attribute allows you to indicate the main language used in a document, but this attribute was kept in HTML only for backwards compatibility with earlier versions of HTML. This attribute has been replaced by the **xml:lang** attribute in new XHTML documents.

Example

```
<!DOCTYPE html>
<html lang = "en">
```

```
<head>
<title>English Language Page</title>
</head>
<body>
This page is using English Language
</body>
</html>
```

2.16 The xml:lang Attribute

The *xml:lang* attribute is the XHTML replacement for the *lang* attribute. The value of the *xml:lang* attribute should be an ISO-639 country code as mentioned in the previous section.

❖ Generic Attributes

Here's a table of some other attributes that are readily usable with many of the HTML tags.

Attribute	Options	Function
align	right, left, center	Horizontally aligned tags
valign	top, middle, bottom	Vertically aligns tags within an HTML element.
bgcolor	numeric, hexadecimal, RGB values	Places a background color behind an element
background	URL	Places a background image behind an element
id	User Defined	Names an element for use with Cascading Style Sheets.
class	User Defined	Classifies an element for use with Cascading Style Sheets.

Table 2: Generic Attributes

CHAPTER 3

CSS-OVERVIEW

CSS is used to control the style of a web document in a simple and easy way.

CSS is the acronym for **"Cascading Style Sheet"**. This tutorial covers both the versions CSS1, CSS2 and CSS3, and gives a complete understanding of CSS, starting from its basics to advanced concepts.

3.1 Why to Learn CSS?

Cascading Style Sheets, fondly referred to as **CSS**, is a simple design language intended to simplify the process of making web pages presentable.

CSS is a MUST for students and working professionals to become a great Software Engineer specially when they are working in Web Development Domain. I will list down some of the key advantages of learning CSS:

- **Create Stunning Web site** - CSS handles the look and feel part of a web page. Using CSS, you can control the color of the text, the style of fonts, the spacing between paragraphs, how columns are sized and laid out, what background images or colors are used, layout designs,variations in display for different devices and screen sizes as well as a variety of other effects.

- **Become a web designer** - If you want to start a carrer as a professional web designer, HTML and CSS designing is a must skill.

- **Control web** - CSS is easy to learn and understand but it provides powerful control over the presentation of an HTML document. Most commonly, CSS is combined with the markup languages HTML or XHTML.

- **Learn other languages** - Once you understands the basic of HTML and CSS then other related technologies like javascript, php, or angular are become easier to understand.

3.2 Hello World using CSS.

Just to give you a little excitement about CSS, I'm going to give you a small conventional CSS Hello World program. You can try it using the Demo link.

```
<!DOCTYPE html>
<html>
<head>
<title>This is document title</title>
<style> h1 {
color: #36C F;
}
</style>
</head>
<body>
<h1>Hello World!</h1>
</body>
</html>
```

3.3 Applications of CSS

As mentioned before, CSS is one of the most widely used style languages over the web. I'm going to list few of them here:

3.3.1 **CSS saves time** - You can write CSS once and then reuse the same sheet in multiple HTML pages. You can define a style for each HTML element and apply it to as many Web pages as you want.

3.3.2 **Pages load faster** - If you are using CSS, you do not need to write HTML tag attributes every time. Just write one CSS rule of a tag and apply it to all the occurrences of that tag. So less code means faster download times.

3.3.3 **Easy maintenance** - To make a global change, simply change the style, and all elements in all the web pages will be updated automatically.

3.3.4 **Superior styles to HTML** - CSS has a much wider array of attributes than HTML, so you can give a far better look to your HTML page in comparison to HTML attributes.

3.3.5 **Multiple Device Compatibility** - Style sheets allow content to be optimized for more than one type of device. By using the same HTML document, different versions of a website can be presented for handheld devices such as PDAs and cell phones or for printing.

3.3.6 **Global web standards** - Now HTML attributes are being deprecated and it is being recommended to use CSS. So it's a good idea to start using CSS in all the HTML pages to make them compatible with future browsers.

3.4 Audience

This **CSS tutorial** will help both students as well as professionals who want to make their websites or personal blogs more attractive.

3.5 Prerequisites

3.5.1 You should be familiar with:

3.5.2 Basic word processing using any text editor.

3.5.3 How to create directories and files.

3.5.4 How to navigate through different directories.

3.5.5 Internet browsing using popular browsers like Internet Explorer or Firefox.

3.5.6 Developing simple Web Pages using HTML or XHTML

3.6 Advantages of CSS

3.6.1 **CSS saves time** − You can write CSS once and then reuse the same sheet in multiple HTML pages. You can define a style for each HTML element and apply it to as many Web pages as you want.

3.6.2 **Pages load faster** − If you are using CSS, you do not need to write HTML tag attributes every time. Just write one CSS rule of a tag and apply it to all the occurrences of that tag. So less code means faster download times.

3.6.3 **Easy maintenance** − To make a global change, simply change the style, and all elements in all the web pages will be updated automatically.

CSS-SYNTAX

A CSS comprises style rules that are interpreted by the browser and then applied to the corresponding elements in your document. A style rule is made of three parts −

3.6.4 **Selector** − A selector is an HTML tag at which a style will be applied. This could be any tag like <h1> or <table> etc.

3.6.5 **Property** − A property is a type of attribute of an HTML tag. Put simply, all the HTML attributes are converted into CSS properties. They could be *color*, *border* etc.

3.6.6 **Value** − Values are assigned to properties. For example, *color* property can have value either *red* or *#F1F1F1* etc.

You can put CSS Style Rule Syntax as follows −

selector { property: value }

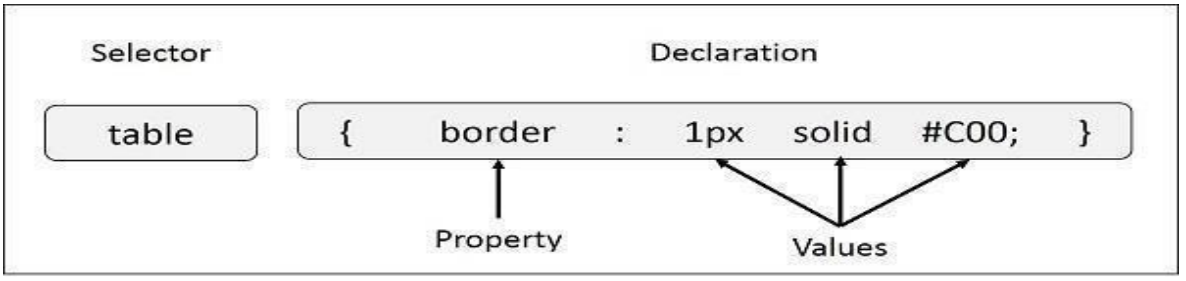


Fig 2 : css syntax

Example − You can define a table border as follows −

table{ border :1px solid #C00; }

Here table is a selector and border is a property and given value *1px solid #C00* is the value of that property.

You can define selectors in various simple ways based on your comfort. Let me put these selectors one by one.

3.7 The Type Selectors

This is the same selector we have seen above. Again, one more example to give a color to all level 1 headings −

h1 {

color: #36C F;

}

3.8 The Universal Selectors

Rather than selecting elements of a specific type, the universal selector quite simply matches the name of any element type −

* {

color: #000000;

}

This rule renders the content of every element in our document in black.

3.9 The Class Selectors

You can define style rules based on the class attribute of the elements. All the elements having that class will be formatted according to the defined rule.

```
.black {
color: #000000;
}
```

This rule renders the content in black for every element with class attribute set to *black* in our document. You can make it a bit more particular. For example −

```
h1.black {
color: #000000;
}
```

This rule renders the content in black for only <h1> elements with class attribute set to *black.*

You can apply more than one class selectors to a given element. Consider the following

example −

```
<p class = "center bold">
This para will be styled by the classes center and bold.
</p>
```

3.10 The ID Selectors

You can define style rules based on the *id* attribute of the elements. All the elements having that *id* will be formatted according to the defined rule.

```
#black {
color: #000000;
}
```

This rule renders the content in black for every element with the id attribute set to *black* in our document. You can make it a bit more particular. For example −

```
h1#black {
color: #000000;
}
```

This rule renders the content in black for only <h1> elements with *id* attribute set to *black*. The true power of *id* selectors is when they are used as the foundation for descendant selectors, For example −

```
#black h2 { color: #000000;
}
```

In this example all level 2 headings will be displayed in black color when those headings will lie within tags having the id attribute set to *black*.

CSS-Inclusion

There are four ways to associate styles with your HTML document. Most commonly used

methods are inline CSS and External CSS.

3.11 Embedded CSS - The <style> Element

ou can put your CSS rules into an HTML document using the <style> element. This tag is placed inside the <head>...</head> tags. Rules defined using this syntax will be applied to all the elements available in the document. Here is the generic syntax −

```
<!DOCTYPE html>
<html>
<head>
<style type = "text/css" media = "all"> body {
background-color: linen;
}
h1 {
color: maroon; margin-left: 40px;
}
</style>
</head>
<body>
<h1>This is a heading</h1>
<p>This is a paragraph.</p>
</body>
</html>
```

3.12 Inline CSS - The *style* Attribute

You can use the style attribute of any HTML element to define style rules. These rules will be applied to that element only. Here is the generic syntax −

```
<element style = "...style rules.">
```

Attributes

Attribute	Value	Description
style	style rules	The value of *style* attribute is a combination of style declarations separated by semicolon (;).

Table 4: style attribute

Example

Following is the example of inline CSS based on the above syntax −

```
<html>
<head>
</head>

<body>
<h1 style = "color:#36C;"> This is inline CSS
</h1>
</body>
</html>
```

3.13 External CSS - The <link> Element

The <link> element can be used to include an external stylesheet file in your HTML document.

An external style sheet is a separate text file with **.css** extension. You define all the Style rules within this text file and then you can include this file in any HTML document using <link> element.

Here is the generic syntax of including external CSS file −

```
<head>
<link type = "text/css" href = "..." media = "..." />
</head>
```

Attributes

Attributes associated with <style> elements are −

href	URL	Specifies the style sheet file having Style rules. This attribute is required.

media	screen tty tv projection handheld print braille aural all	Specifies the device the document will be displayed on. Default value is *all*. This is an optional attribute.

Table 5: link attribute

Example

Consider a simple style sheet file with a name *mystyle.css* having the following rules −

```
h1, h2, h3 {
color: #36C;
font-weight: normal;
letter-spacing:
.4em; margin-bottom:
1em;
text-transform: lowercase;
} Now you can include this file mystyle.css in any HTML
document as follows −
<head>
<link type = "text/css" href = "mystyle.css" media = " all" />
</head>
```

CHAPTER 4

JAVASCRIPT-OVERVIEW

JavaScript is a lightweight, interpreted **programming** language. It is designed for creating network-centric applications. It is complementary to and integrated with Java. **JavaScript** is very easy to implement because it is integrated with HTML. It is open and cross-platform.

4.1. Why to Learn Javascript

Javascript is a MUST for students and working professionals to become a great Software Engineer specially when they are working in Web Development Domain. I will list down some of the key advantages of learning Javascript:

- Javascript is the most popular **programming language** in the world and that makes it a programmer's great choice. Once you learn Javascript, it helps you develop great front-end as well as back-end softwares using different Javascript based frameworks like jQuery, Node.JS etc.

- Javascript is everywhere, it comes installed on every modern web browser and so to learn Javascript you really do not need any special environment setup. For example Chrome, Mozilla Firefox , Safari and every browser you know as of today, supports Javascript.

- Javascript helps you create really beautiful and crazy fast websites. You can develop your website with a console like look and feel and give your users the best Graphical User Experience.

- JavaScript usage has now extended to mobile app development, desktop app development, and game development. This opens many opportunities for you as a Javascript Programmer.

- Due to high demand, there is tons of job growth and high pay for those who know JavaScript. You can navigate over to different job sites to see what having JavaScript skills looks like in the job market.

- Great thing about Javascript is that you will find tons of frameworks and Libraries already developed which can be used directly in your software development too.

There could be 1000s of good reasons to learn Javascript Programming. But one thing for sure, to learn any **programming language**, not only Javascript, you just need to code, and code and finally code until you become an expert.

4.2. Hello World using Javascript

Just to give you a little excitement about **Javascript programming**, I'm going to give you a small conventional Javascript Hello World program.

```
<html>
<body>
<script language = "javascript" type = "text/javascript">
<!--
document.write("Hello World!")
//-->
</script>
</body>
</html>
```

There are many useful **Javascript frameworks** and libraries available:

- Angular
- React
- jQuery
- Vue.js
- Ext.js
- Ember.js
- Meteor

- Mithril
- Node.js
- Polymer
- Aurelia
- Backbone.js

It is really impossible to give a complete list of all the available Javascript frameworks and libraries. The Javascript world is just too large and too much new is happening.

4.3. Applications of Javascript Programming

As mentioned before, **Javascript** is one of the most widely used **programming languages** (Front-end as well as Back-end). It has it's presence in almost every area of software development. I'm going to list few of them here:

4.3.1. **Client side validation** - This is really important to verify any user input before submitting it to the server and Javascript plays an important role in validating those inputs at the front-end itself.

4.3.2. **Manipulating HTML Pages** - Javascript helps in manipulating HTML pages on the fly. This helps in adding and deleting any HTML tag very easily using javascript and modifying your HTML to change its look and feel based on different devices and requirements.

4.3.3. **User Notifications** - You can use Javascript to raise dynamic pop-ups on the webpages to give different types of notifications to your website visitors.

4.3.4. **Back-end Data Loading** - Javascript provides Ajax library which helps in loading back-end data while you are doing some other processing. This really gives an amazing experience to your website visitors.

This list goes on, there are various areas where millions of software developers are happily using Javascript to develop great websites and other softwares.

4.4. Prerequisites

For this **Javascript tutorial**, it is assumed that the reader has prior knowledge of HTML coding. It would help if the reader had some prior exposure to object-oriented programming concepts and a general idea on creating online applications.

4.5. Client-Side JavaScript

Client-side JavaScript is the most common form of the language. The script should be included in or referenced by an HTML document for the code to be interpreted by the browser.

It means that a web page need not be a static HTML, but can include programs that interact with the user, control the browser, and dynamically create HTML content.

The JavaScript client-side mechanism provides many advantages over traditional CGI server-side scripts. For example, you might use JavaScript to check if the user has entered a valid email address in a form field.

The JavaScript code is executed when the user submits the form, and only if all the entries are valid, they would be submitted to the Web Server.

JavaScript can be used to trap user-initiated events such as button clicks, link navigation, and other actions that the user initiates explicitly or implicitly.

4.6. Advantages of JavaScript

The merits of using JavaScript are −

4.6.1. **Less server interaction** − You can validate user input before sending the page off to the server. This saves server traffic, which means less load on your server.

4.6.2. **Immediate feedback to the visitors** − They don't have to wait for a page reload to see if they have forgotten to enter something.

4.6.3. **Increased interactivity** − You can create interfaces that react when the user hovers over them with a mouse or activates them via the keyboard.

CHAPTER 5

NODE.JS

Node JS is an open-source and cross-platform runtime environment built on Chrome's V8 JavaScript engine for executing JavaScript code outside of a browser. It provides an event-driven, non-blocking (asynchronous) I/O and cross-platform runtime environment for building highly scalable server-side applications using JavaScript.

5.1 Why to learn Node.Js?

1. In today's development field, learning JavaScript is indispensable. Whether you're building a dynamic website, a mobile app, or a progressive web application, JavaScript is a critical skill since it dominates the front-end development landscape. Instead of learning other backend technologies like PHP, Java, or Ruby, mastering Node.js allows developers to use JavaScript for both front-end and back-end development, creating a seamless development experience with a single programming language across the entire stack.

2. Node.js has emerged as one of the most sought-after technologies worldwide, particularly in tech hubs like Silicon Valley. Its asynchronous, event-driven architecture makes it highly efficient for building scalable, real-time applications. Node.js is also backed by a vast ecosystem of open-source libraries via npm (Node Package Manager), which accelerates development and ensures easy access to solutions for common challenges.

3. For developers aspiring to build cutting-edge applications, Node.js is ideal for creating RESTful APIs, handling real-time data with WebSockets, and managing microservices architectures. Its ability to handle a large number of simultaneous connections makes it a popular choice for companies ranging from startups to tech giants like Netflix, LinkedIn, and PayPal.

4. By learning Node.js, software developers can open the doors to countless career opportunities. Its widespread demand, modern architecture, and capability to integrate seamlessly with other emerging technologies like Docker, Kubernetes, and

cloud platforms make it a perfect skill to future-proof your career and stay competitive in the evolving tech landscape.

5.2 Features of Node.Js

5.2.1 **Easy Scalability:** Node JS is built upon Chrome V8's engine powered by Google. It allows Node to provide a server-side runtime environment that compiles and executes JavaScript at lightning speeds.

5.2.2 **Real time web apps:** Today the web has become much more about interaction. Users want to interact with each other in real-time. Chat, gaming, constant social media updates, collaboration tools, eCommerce websites, real-time tracking apps, marketplace- each of these features requires real-time communication between users, clients, and servers across the web.

5.2.3 **Fast Suite:** As we have discussed, NodeJS is highly scalable and lightweight that's why it's a heavy favorite for microservice architectures. In a nutshell, microservice architectures mean breaking down the application into isolated and independent services.

5.2.4 **Easy to learn and code:** No matter what language you are using for the backend application you're gonna need JavaScript for front-end anyway so instead of spending your time learning a server-side language such as Php, Java or Ruby on Rails, you can spend all your efforts in learning JS and mastering in it.

5.2.5 **Data Streaming:** NodeJS comes to the rescue since it's good at handling such an I/O process which allows users to transcode media files simultaneously while they are being uploaded. It takes less time compared to other data processing methods for processing data.

5.2.6 **Corporate Support:** It's an independent community aimed at facilitating the development of NodeJS core tools. The foundation of NodeJS was formed to speed up the development of NodeJS, and it was intended to allow broad adoption of it.

5.3 Concepts of Node.js

The following diagram depicts some important parts of Node.js that are useful and help us

understand it better.

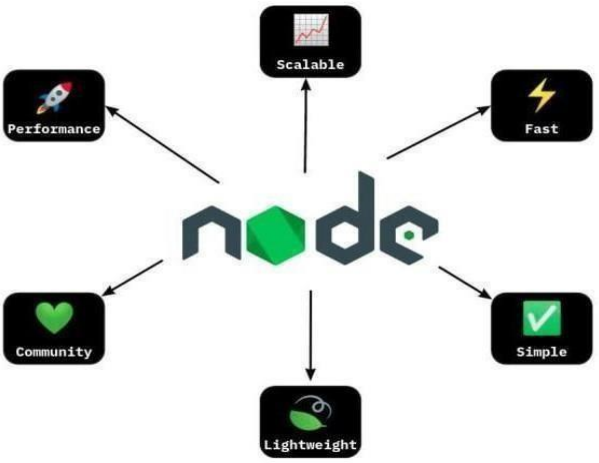


Fig 3: concepts of Node.js

5.4 How does Node.js work?

Node.js accepts the request from the clients and sends the response, while working with the request node.js handles them with a single thread. To operate I/O operations or requests node.js use the concept of threads. Thread is a sequence of instructions that the server needs to perform. It runs parallel on the server to provide the information to multiple clients. Node.js is an event loop single-threaded language. It can handle concurrent requests with a single thread without blocking it for one request.

5.5 Advantages of NodeJS

1. **Easy Scalability:** Easily scalable the application in both horizontal and vertical directions.
2. **Real-time web apps:** Node.js is much more preferable because of faster synchronization. Also, the event loop avoids HTTP overloaded for Node.js development.
3. **Fast Suite:** NodeJS acts like a fast suite and all the operations can be done quickly like reading or writing in the database, network connection, or file system
4. **Easy to learn and code:** NodeJS is easy to learn and code because it uses JavaScript.
5. **Advantage of Caching:** It provides the caching of a single module. Whenever there is any request for the first module, it gets cached in the application memory, so you don't need to re-execute the code.
6. **Data Streaming:** In NodeJS HTTP request and response are considered as two

separate events.

7. **Hosting:** PaaS (Platform as a Service) and Heroku are the hosting platforms for NodeJS application deployment which is easy to use without facing any issue.
8. **Corporate Support:** Most of the well-known companies like Walmart, Paypal, Microsoft, Yahoo are using NodeJS for building the applications.

5.6 Reason to Choose Node.js

There are other programming languages also which we can use to build back-end services so what makes Node.js different I am going to explain.

1. Ease of Use and Rapid Prototyping

Node.js is beginner-friendly and easy to get started with, making it an excellent choice for both novice and experienced developers. Its simplicity enables rapid prototyping, allowing developers to build and test ideas quickly. This feature is particularly valuable in agile development environments, where speed and adaptability are critical.

2. High Performance and Scalability

Node.js leverages Google's V8 JavaScript engine, which compiles JavaScript into highly optimized machine code, delivering exceptional performance. Its non-blocking, event-driven architecture allows it to handle multiple concurrent requests efficiently, making it ideal for building fast, scalable services and applications.

3. JavaScript Everywhere

With Node.js, developers can use JavaScript for both client-side and server-side programming. This unification reduces the need to switch between programming languages, simplifies the development process, and improves collaboration within teams. It also means that JavaScript developers can easily transition into back-end development without learning a completely new language.

4. Extensive Ecosystem and Asynchronous Nature

Node.js boasts a vast ecosystem of open-source libraries and modules accessible through npm (Node Package Manager). These libraries enable developers to add features and functionalities to their applications quickly, saving time and effort. Additionally, Node.js's asynchronous, non-blocking nature ensures that the application remains responsive even under heavy load, making it suitable for real-time applications like chat apps, online gaming, and live streaming.

5. **Community Support and Industry Adoption**

Node.js has a strong and active community that provides continuous support, tutorials, and tools to developers. Its popularity has led to widespread adoption by industry leaders such as Netflix, Uber, and Walmart, further solidifying its credibility and relevance in the tech industry.

5.7 Front-End

```
    return () => clearInterval(timer);
  }, []);

  const sectionVariants = {
    hidden: { opacity: 0, y: 20 },
    visible: {
      opacity: 1,
      y: 0,
      transition: {
        duration: 0.8,
        ease: "easeInOut",
      },
    },
  };

  const hoverEffect = {
    hover: {
      scale: 1.2, // Increased scale for more focus
      boxShadow: "0px 8px 30px rgba(0, 0, 0, 0.3)", // More prominent shadow
      transition: {
        duration: 0.3,
        ease: "easeInOut",
      },
    },
  };

  return (
    <>
      <motion.div initial="hidden" animate="visible" variants={sectionVariants}>
        <NavBar />
      </motion.div>
      <section className="bg-white min-h-screen h-full dark:bg-gray-900 pt-20">
        <motion.div className="flex flex-row justify-between items-center p-8 space-x-4 flex-nowrap" initial="hidden" animate="visible" variants={sectio
          {/* logo */}
          <motion.img
            className="h-20 w-40 rounded-full"
```

Fig: 5.7

5.8 Back-End

```
      const extractedText = response.data.text;
      setMessage(extractedText);
    } catch (error) {
      console.error('Error extracting text from PDF:', error);
      setError('Failed to extract text from PDF. Please try again.');
    }
  }
};

const handlePlayVoice = (text: string) => {
  if (isPlaying && currentUtterance) {
    speechSynthesis.cancel();
    setIsPlaying(false);
    setHighlightedText(null);
    return;
  }

  const utterance = new SpeechSynthesisUtterance(text);
  utterance.onstart = () => {
    setIsPlaying(true);
    setHighlightedText(text);
  };
  utterance.onend = () => {
    setIsPlaying(false);
    setHighlightedText(null);
  };
  utterance.onboundary = (event) => {
    const { charIndex } = event;
    setHighlightedText(text.substring(0, charIndex));
  };

  setCurrentUtterance(utterance);
  speechSynthesis.speak(utterance);
};
```

Fig: 5.8

```
44
45      app.add_middleware(
46          CORSMiddleware,
47          allow_origins=["https://miningniti.vercel.app"],
48          allow_credentials=True,
49          allow_methods=["*"],
50          allow_headers=["*"],
51      )
52
53      class Item(BaseModel):
54          input_query: str
55          source: str  # "database", "internet", or "both"
56
57      @app.post("/chat")
58      def run(item: Item, model: MyChatBot = Depends(lambda: model)):
59          """
60          Endpoint to handle chat requests.
61          """
62          try:
63              if item.source == "database":
64                  results = search_pdf_text(item.input_query)
65                  response = "\n".join([f"PDF: {res['pdf_name']}\nText: {res['text']}" for res in results])
66              elif item.source == "internet":
67                  response = model.run(input=item.input_query)
68              else:  # both
69                  results = search_pdf_text(item.input_query)
70                  db_response = "\n".join([f"PDF: {res['pdf_name']}\nText: {res['text']}" for res in result
71                  internet_response = model.run(input=item.input_query)
72                  response = f"Database Results:\n{db_response}\n\nInternet Results:\n{internet_response}"
73              return JSONResponse(content={"response": response})
74          except Exception as e:
75              raise HTTPException(status_code=500, detail=str(e))
76
```

Project Overview: Logikendra – A Comprehensive Logistics Management System

Introduction: Logikendra is a robust logistics management system developed to optimize

and streamline the document and data management processes at **Om Logistics Ltd**. The system leverages the **MERN stack** (MongoDB, Express.js, React, Node.js) to provide a centralized platform where logistics-related documents can be efficiently stored, accessed, and managed. This project represents a significant step forward in enhancing the operational capabilities of the logistics company through advanced technologies.

Objective: The primary goal of **Logikendra** is to create a seamless and efficient system that centralizes document management, improves retrieval processes, and supports better decision-making. By deploying advanced technologies and intuitive user interfaces, the system is designed to handle large volumes of documents and optimize their management.

Project Components:

1. **Frontend Development:**

❖ **Technology Used**: React.js and Next.js for building a user-friendly interface.

❖ **Features**:

- Intuitive navigation for uploading, viewing, and searching documents.
- Real-time updates and responsive design to ensure a smooth user experience across devices.

❖ **Purpose**: Simplifies the interaction between users and the backend, allowing quick document access and task management.

2. **Backend Development:**

❖ **Technology Used**: Node.js with Express.js for server-side logic and API development.

❖ **Key Functionalities**:

- **User Authentication**: Implemented using **JSON Web Tokens (JWT)** to ensure secure access and role-based permissions.
- **Document Management**: APIs for uploading, categorizing, and retrieving documents stored in MongoDB.
- **Search Mechanism**: Integration of **FAISS (Facebook AI Similarity Search)** for advanced, efficient document searches using vector-based retrieval techniques.

❖ **Middleware**: Custom middleware for request validation, authentication, and error

handling.

3. **Database Design:**

❖ **Database Used**: MongoDB, chosen for its scalability and schema-less structure.

❖ **Schema Design**:

- User Schema: Manages user data, including authentication and roles.
- Document Schema: Stores metadata related to uploaded documents, such as type, category, and timestamps.

4. **Deployment Strategy:**

❖ **Frontend Deployment**: Hosted on **Vercel** for high availability and performance.

❖ **Backend Deployment**: Hosted on **Render**, ensuring scalability and continuous deployment.

❖ **Version Control and CI/CD**: Employed **Git** and basic CI/CD practices for seamless updates and version management.

Core Features:

- **User Authentication**: Secure login and access management using **JWT**.
- **Document Upload and Retrieval**: Efficient uploading and retrieval of documents with search filters and advanced categorization.
- **Search Functionality**: Implemented using **FAISS** for near-instantaneous search capabilities, allowing users to find documents quickly and accurately.
- **Role-Based Access Control**: Ensures that only authorized personnel can access specific features and data.

Challenges and Solutions

When building robust systems, developers often face several challenges. Overcoming these obstacles not only ensures better performance and security but also lays the foundation for future scalability. Here are some key challenges encountered and the solutions implemented:

1. **Scalability Issues**

   As user demand grows, scalability can become a major challenge, especially for systems handling large volumes of data and concurrent requests. This was addressed

by implementing database indexing to optimize query performance and by designing efficient API endpoints to reduce response times. Additionally, horizontal scaling strategies, such as load balancing and server clustering, were considered to ensure the system could handle an increasing number of users without performance degradation.

2. **Security Concerns**

   Security is paramount, especially when dealing with sensitive data. Robust measures were implemented, including secure user authentication mechanisms, such as multi-factor authentication (MFA), to protect user accounts. Data was safeguarded using end-to-end encryption for both storage and transmission, ensuring that unauthorized access or breaches were minimized. Regular security audits and adherence to best practices further strengthened the system's defenses.

3. **Data Retrieval Speed**

   Retrieving data quickly and efficiently is critical for maintaining a seamless user experience. The integration of FAISS (Facebook AI Similarity Search) significantly improved search response times, particularly for large datasets. This allowed for rapid similarity searches and optimized the overall performance of the system. Cache mechanisms, such as Redis, were also explored to further enhance data retrieval speed for frequently accessed information.

Future Enhancements

While the current system meets the requirements, there is significant potential for further improvements and feature additions to make it more advanced and user-friendly. These include:

1. **AI-PoweredSearch**

   Incorporating machine learning algorithms into the search functionality can enhance user experience by providing more accurate predictions and personalized recommendations. For instance, natural language processing (NLP) models can help interpret complex queries and deliver context-aware results, improving search accuracy and efficiency.

2. **Mobile Application**

   To make the system more accessible, a dedicated mobile application can be developed. This would enable users to interact with the system seamlessly from their smartphones or tablets, ensuring usability on the go. By leveraging modern frameworks like React Native or Flutter, a cross-platform solution could be built

to support both Android and iOS devices efficiently.

3. **PredictiveAnalytics**

   Harnessing the power of data analytics and machine learning, the system can predict logistics trends and provide insights that aid decision-making. Predictive models can be used to identify patterns, forecast demand, optimize resource allocation, and enhance overall operational efficiency. This feature would be especially valuable for businesses looking to stay ahead in competitive markets.

CHAPTER 6

Conclusion

The industrial training in web development has been an invaluable learning experience, providing me with practical exposure to real-world projects, such as **Logikendra**, a comprehensive logistics management system developed for **Om Logistics Ltd**. This project, built using the **MERN stack**, enabled me to apply academic knowledge, tackle complex problems, and develop innovative solutions. The hands-on experience enhanced my technical proficiency, problem-solving abilities, and understanding of scalable system development.

Collaborating with experienced professionals offered valuable mentorship and industry insights, reinforcing the importance of teamwork, resilience, and effective communication. The exposure to modern web technologies and tools broadened my skill set, preparing me for diverse project requirements and the dynamic nature of web development. The challenges faced and constructive feedback received taught me perseverance, efficient project management, and adaptability.

Overall, this training has significantly prepared me to contribute confidently to future web development projects and pursue a successful career in the field. I am grateful to the company and mentors who supported me throughout this journey and look forward to leveraging the skills and knowledge gained in my future endeavors.

Chapter 7

Reference

[1] **Bocutiu, C.** (2021). *Full Stack Web Development with MongoDB, Express, React, and Node*. Packt Publishing. This book provides an in-depth guide to building modern web applications using the complete MERN stack.

[2] **Brown, E., & Kumar, R.** (2020). *Learning React: Modern Patterns for Developing React Apps*. O'Reilly Media. This book explains the core concepts of React and how to build applications with reusable components, a crucial part of MERN stack development.

[3] **Multer, D.** (2022). *Node.js Design Patterns: Design and implement production-grade Node.js applications using proven patterns and techniques*. Packt Publishing. This reference focuses on best practices for developing scalable server-side applications using Node.js and Express.

[4] **MongoDB Inc.** (2024). *MongoDB Documentation*. Available at: https://www.mongodb.com/docs/.

[5] **Express.js Team** (2024). *Express.js Guide: API Reference and Best Practices*. Available at: https://expressjs.com/. This official guide provides comprehensive knowledge on Express.js framework for building server-side applications.

[6] **Jackson, C.**, & **Noviello, B.** (2020). *React and TypeScript: Develop maintainable and scalable web applications using TypeScript, React, and Redux*. Packt Publishing. Offers insights into integrating TypeScript with React for better type safety in full stack development.

[7] **Node.js Foundation** (2024). *Node.js API Documentation*. Available at: https://nodejs.org/en/docs/. A crucial resource for understanding the built-in modules, event-driven architecture, and features of Node.js.

[8] **Hart, A.** (2019). *Full-Stack React Projects: Modern web development using React, Node, Express, and MongoDB*. Packt Publishing.

[9] **MDN Web Docs** (2024). *JavaScript Guide*. Available at: https://developer.mozilla.org/en-US/docs/Web/JavaScript. This guide is essential for understanding core JavaScript concepts used across all layers of the MERN stack.

[10] **Chinnathambi, A.** (2017). *Pro MERN Stack: Full Stack Web App Development with Mongo, Express, React, and Node*. Apress.